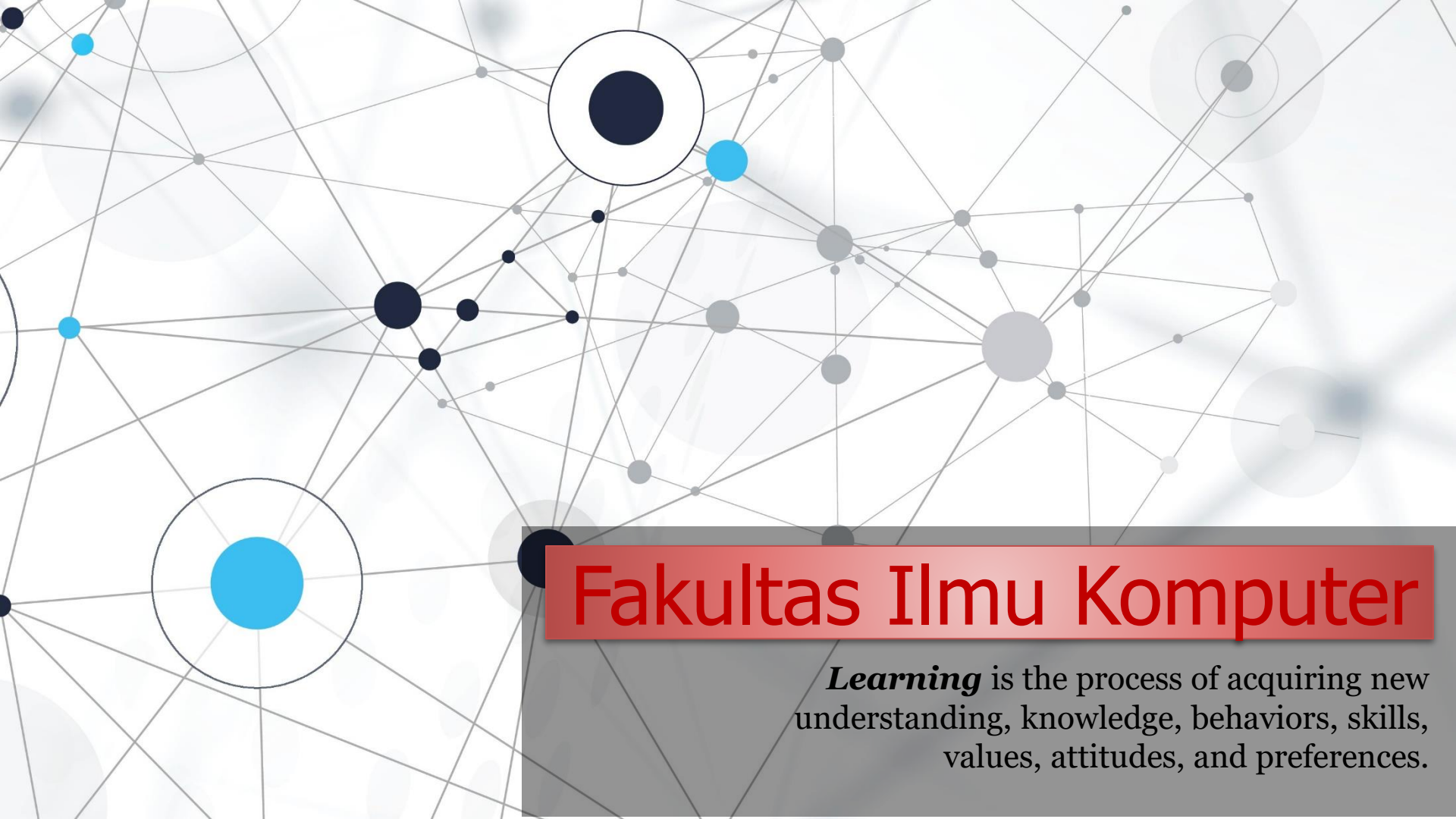




Fakultas Ilmu Komputer

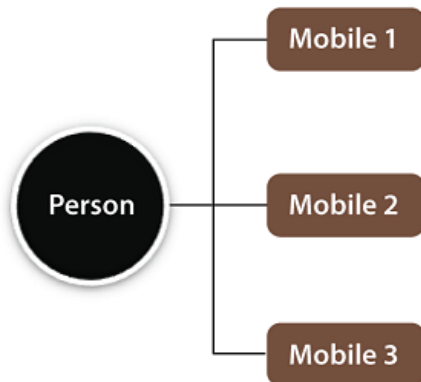
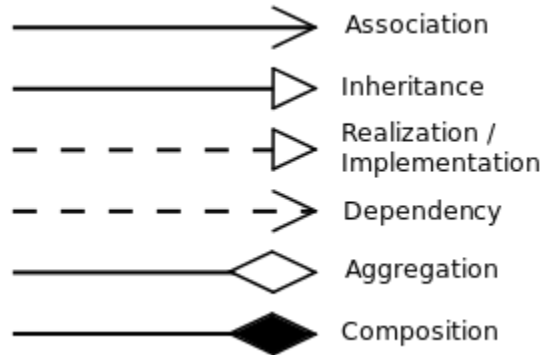
Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

Class Activity #3

Association *in* Python OOP



Types of Associations in OOP



- 1) Asosiasi biasa (*ordinary association*): objek memiliki hubungan yang terdefinisi secara jelas dan objek tersebut tidak bergantung satu sama lain.
- 2) Agregasi (*aggregation*): satu objek memiliki beberapa objek terkait, tetapi objek-objek tersebut dapat ada tanpa objek utama.
- 3) Komposisi (*composition*): satu objek terdiri dari beberapa objek terkait, dan objek-objek tersebut hanya dapat ada dengan objek utama.
- 4) Inheritance (*pewarisan*): objek turunan mewarisi sifat dan perilaku dari objek induk.

Penggunaan asosiasi yang tepat dapat membantu mengorganisir kode program dan membuat program lebih mudah dipahami dan dikembangkan.

Association Relationship *with* Python

Example association relationship using Python.

```
# class 1
class Book:
    # constructor with parameters
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn

    def __str__(self):
        return f"{self.title} by {self.author}."
```

(1)

```
# class 2
class Library:
    # constructor without parameter
    def __init__(self):
        self.books = []

    # fungsi menambah buku
    def addBook(self, book):
        print("Added this book: "+book.__str__())
        self.books.append(book)

    # fungsi menghapus buku
    def removeBook(self, book):
        self.books.remove(book)

    # fungsi mencari buku berdasarkan judul
    def searchBook(self, title):
        for book in self.books:
            if book.title == title:
                return book
        return None
```

(2)

```
# class 3
class Student:
    # constructor with parameter
    def __init__(self, name):
        self.name = name
        self.borrowed_book = None
        print(f"{self.name} has been registered.")

    # fungsi meminjam buku
    def borrowBook(self, book):
        if self.borrowed_book is None:
            self.borrowed_book = book
            print(f"{self.name} has borrowed {book.title}.")
        else:
            print(f"{self.name} has already borrowed {self.borrowed_book.title}.")

    # fungsi mengembalikan buku
    def returnBook(self):
        if self.borrowed_book is not None:
            book = self.borrowed_book
            self.borrowed_book = None
            print(f"{self.name} has returned {book.title}.")
        else:
            print(f"{self.name} has not borrowed any book.")

    # fungsi menampilkan buku yang dipinjam
    def viewBorrowedBooks(self):
        if self.borrowed_book is not None:
            print(f"{self.name} has borrowed {self.borrowed_book.title}.")
        else:
            print(f"{self.name} has not borrowed any book.")
```

(3)

Implementasi Asosiasi Classes:

```
# create objek library
unklab_library = Library()

print("\nINPUT BUKU KE PERPUSTAKAAN:");
print("=====");
# add beberapa buku ke perpustakaan
book1 = Book("Python Programming", "Semmy Taju", "123456")
book2 = Book("Java Programming", "Green Sandag", "234567")
book3 = Book("C++ Programming", "Patricia Eugenie", "345678")
unklab_library.addBook(book1)
unklab_library.addBook(book2)
unklab_library.addBook(book3)

print("\nMAHASISWA DAFTAR DI PERPUSTAKAAN:");
print("=====");
# creat objek mahasiswa
student1 = Student("Kadek")
student2 = Student("Jeremy")
student3 = Student("Ellen")
student4 = Student("Bobby")
student5 = Student("Putra")
```

```
print("\nPROSES PINJAM BUKU DI PERPUSTAKAAN:");
print("=====");
# meminjam buku
student1.borrowBook(book1)
student2.borrowBook(book2)
student2.borrowBook(book3) # tidak bisa meminjam lebih dari satu buku.
student3.borrowBook(book2)
student4.borrowBook(book3)
student5.borrowBook(book3)

# view buku yang dipinjam
student1.viewBorrowedBooks()
student2.viewBorrowedBooks()

# mengembalikan buku
student1.returnBook()
student2.returnBook()

# view buku yang dipinjam setelah dikembalikan
student1.viewBorrowedBooks()
student2.viewBorrowedBooks()
```

Pilih tipe hubungan yang tepat:

- 1) One-to-one association ?
- 2) One-to-many association ?
- 3) Many-to-many association ?

Expected Output

```
INPUT BUKU KE PERPUSTAKAAN:
=====
Added this book: Python Programming by Semmy Taju.
Added this book: Java Programming by Green Sandag.
Added this book: C++ Programming by Patricia Eugenie.

MAHASISWA DAFTAR DI PERPUSTAKAAN:
=====
Kadek has been registered.
Jeremy has been registered.
Ellen has been registered.
Bobby has been registered.
Putra has been registered.

PROSES PINJAM BUKU DI PERPUSTAKAAN:
=====
Kadek has borrowed Python Programming.
Jeremy has borrowed Java Programming.
Jeremy has already borrowed Java Programming.
Ellen has borrowed Java Programming.
Bobby has borrowed C++ Programming.
Putra has borrowed C++ Programming.
Kadek has borrowed Python Programming.
Jeremy has borrowed Java Programming.
Kadek has returned Python Programming.
Jeremy has returned Java Programming.
Kadek has not borrowed any book.
Jeremy has not borrowed any book.
```

To do list:

- 1) Tambahkan 10 buku baru ke dalam perpustakaan, sehingga total buku yang dapat di pinjam adalah 13 buku.
- 2) Tambahkan 10 mahasiswa baru yang akan daftar ke perpustakaan Unklab.
- 3) Pastikan bahwa semua mahasiswa meminjam sebuah buku.

One-to-one Association Relationship

One-to-one association relationship using Python.

```
# class 1
class Employee:
    # constructor with parameters
    def __init__(self, name, age):
        self.name = name
        self.age = age
        self.companies = []

    def add_company(self, company):
        self.companies.append(company)

    def remove_company(self, company):
        self.companies.remove(company)

# class 2
class Company:
    # constructor with parameters
    def __init__(self, name):
        self.name = name
        self.employees = []

    def add_employee(self, employee):
        if employee not in self.employees:
            self.employees.append(employee)
            employee.add_company(self)

    def remove_employee(self, employee):
        if employee in self.employees:
            self.employees.remove(employee)
            employee.remove_company(self)
```

```
# membuat objek perusahaan
company = Company("PT Budi Mulya")

# membuat objek karyawan
employee1 = Employee("Semmy Taju", 25)
employee2 = Employee("Ferdy Mesakh", 30)

# menambahkan karyawan ke perusahaan
company.add_employee(employee1)
company.add_employee(employee2)

# mencetak nama karyawan dan nama perusahaan tempatnya bekerja
print("Karyawan 1: ", employee1.name)
for c in employee1.companies:
    print("Perusahaan 1: ", c.name)
print("Karyawan 2: ", employee2.name)
for c in employee2.companies:
    print("Perusahaan 2: ", c.name)

# mencetak nama perusahaan dan nama karyawan yang bekerja di dalamnya
print("Perusahaan: ", company.name)
for e in company.employees:
    print("Karyawan: ", e.name)

# menghapus karyawan dari perusahaan
company.remove_employee(employee1)

# mencetak nama karyawan dan nama perusahaan setelah dihapus
print("Karyawan 1: ", employee1.name)
for c in employee1.companies:
    print("Perusahaan 1: ", c.name)
print("Karyawan 2: ", employee2.name)
for c in employee2.companies:
    print("Perusahaan 2: ", c.name)
print("Perusahaan: ", company.name)
for e in company.employees:
    print("Karyawan: ", e.name)
```

(1)

(Many)

Implementasi one-to-one association dilakukan dengan cara menyimpan objek perusahaan di dalam objek karyawan (atribut **company**), dan objek karyawan di dalam objek perusahaan (atribut **employee**). **Kedua objek saling mereferensikan satu sama lain.**

Contoh program diatas termasuk
pada ***one-to-one association***
relationship atau ***one-to-many***
association relationship ??
Jelaskan pendapat anda?



Jawabannya adalah one-to-many association relationship, karena satu object ***company*** bisa memiliki lebih dari satu object ***employee***.
Bagaimana cara supaya bisa menjadi **one-to-one association relationship** saja??



One-to-many Association Relationship

One-to-many association relationship using Python.

```
# class 1
class Zoo:
    # constructor with parameters
    def __init__(self, name):
        self.name = name
        self.animals = []

    def add_animal(self, animal):
        self.animals.append(animal)
        animal.set_zoo(self)

    def remove_animal(self, animal):
        self.animals.remove(animal)
        animal.set_zoo(None)
        print(f"Animal {animal.name} dikeluarkan/dihapus.");

# class 2
class Animal:
    # constructor with parameters
    def __init__(self, name, species):
        self.name = name
        self.species = species
        self.zoo = None

    def set_zoo(self, zoo):
        self.zoo = zoo

    def get_zoo(self):
        return self.zoo.name if self.zoo else None
```

```
# membuat objek zoo
zoo = Zoo("Taman Safari Jakarta")

# membuat objek animal
animal1 = Animal("Kuda", "Equus caballus")
animal2 = Animal("Jerapah", "Giraffa camelopardalis")
animal3 = Animal("Harimau", "Panthera tigris")

# menambahkan animal ke zoo
zoo.add_animal(animal1)
zoo.add_animal(animal2)
zoo.add_animal(animal3)

# mencetak daftar animal di zoo
print("Animal di", zoo.name, ":")
for animal in zoo.animals:
    print(animal.name, "(" + animal.species + ")")

# mencetak zoo tempat animal tinggal
print(animal1.name, "tinggal di", animal1.get_zoo())
print(animal2.name, "tinggal di", animal2.get_zoo())
print(animal3.name, "tinggal di", animal3.get_zoo())

# menghapus animal dari zoo
zoo.remove_animal(animal2)
```



- ❖ Disebut **one-to-many association** karena satu objek **Zoo** dapat memiliki banyak objek **Animal** yang tinggal di kebun binatang.
- ❖ Dalam hubungan ini, satu objek dari class Zoo dapat memiliki banyak objek dari class Animal, namun satu objek dari class Animal hanya bisa dimiliki oleh satu objek dari class Zoo (dalam hal ini, satu objek dari class Animal hanya bisa tinggal di satu objek Zoo).

Many-to-many Association Relationship

Many-to-many association relationship using Python.

```
# class 1
class Animal:
    # constructor with parameters
    def __init__(self, name, species):
        self.name = name
        self.species = species
        self.habitats = []

    def add_habitat(self, habitat):
        if habitat not in self.habitats:
            self.habitats.append(habitat)
            habitat.add_animal(self)

    def remove_habitat(self, habitat):
        if habitat in self.habitats:
            self.habitats.remove(habitat)
            habitat.remove_animal(self)

# class 2
class Habitat:
    # constructor with parameters
    def __init__(self, name):
        self.name = name
        self.animals = []

    def add_animal(self, animal):
        if animal not in self.animals:
            self.animals.append(animal)
            animal.add_habitat(self)

    def remove_animal(self, animal):
        if animal in self.animals:
            self.animals.remove(animal)
            animal.remove_habitat(self)
```

```
# create animal objects
lion = Animal("Simba", "Lion")
tiger = Animal("Rajah", "Tiger")
monkey = Animal("Abu", "Monkey")

# create habitat objects
savanna = Habitat("Savanna") # padang rumput
jungle = Habitat("Jungle") # hutan

# add habitats for animals
lion.add_habitat(savanna)
lion.add_habitat(jungle)
tiger.add_habitat(jungle)
monkey.add_habitat(jungle)

# print animals and their habitats
print(lion.name + " lives in: ")
for habitat in lion.habitats:
    print("- " + habitat.name)

print(tiger.name + " lives in: ")
for habitat in tiger.habitats:
    print("- " + habitat.name)

print(monkey.name + " lives in: ")
for habitat in monkey.habitats:
    print("- " + habitat.name)

# print habitats and their animals
print(savanna.name + " is home to: ")
for animal in savanna.animals:
    print("- " + animal.name + ", " + animal.species)

print(jungle.name + " is home to: ")
for animal in jungle.animals:
    print("- " + animal.name + ", " + animal.species)
```

Class di atas disebut sebagai **many-to-many association** karena objek **Animal** dapat memiliki beberapa objek **Habitat** dan sebaliknya, objek **Habitat** juga dapat memiliki beberapa objek **Animal**.

- A. Buatlah semua code program yang ada di dalam ppt file ini.
- B. Buat report untuk membeda type hubungan asosiasi berikut ini:
 - 1) one-to-one association
 - 2) one-to-many association
 - 3) many-to-many association
- C. Screenshot gambar dan berikan penjelasan pada setiap gambar.
- D. Submit dalam format PDF.

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

